

Análisis Comparativo de Arquitecturas RAG y Ajuste Fino en Modelos de Lenguaje Compactos para Lenguas de Bajos Recursos: Implicaciones para la Educación Intercultural Bilingüe en Quechua Collao

El desarrollo de sistemas de inteligencia artificial generativa para entornos con conectividad severamente limitada y hardware restringido representa uno de los desafíos más formidables en la ingeniería de software y el Procesamiento de Lenguaje Natural (PLN) contemporáneo. En el contexto de las comunidades rurales del sur andino, la brecha de infraestructura digital impone una restricción ineludible: más del 79% de los hogares carecen de acceso a internet, lo que hace técnica y operativamente inviable la dependencia de modelos comerciales de frontera alojados en la nube¹. Ante esta realidad estructural, el despliegue local de Modelos de Lenguaje Compactos (Small Language Models o SLMs), típicamente de menos de 10 mil millones de parámetros, emerge como la única vía factible para democratizar el acceso a interfaces conversacionales avanzadas en la Educación Intercultural Bilingüe (EIB)¹.

La transición desde Grandes Modelos de Lenguaje (LLMs) que operan en clústeres masivos de procesamiento tensorial hacia SLMs ejecutables en procesadores centrales (CPU) exige reconsiderar por completo cómo se inyecta, actualiza y gestiona el conocimiento especializado². En el procesamiento de lenguas de bajos recursos digitales (low-resource languages) como el Quechua Collao, el modelo pre-entrenado base carece de representaciones paramétricas profundas sobre la sintaxis específica, el léxico curricular y la pertinencia cultural de la región andina¹. Consecuentemente, surge una disyuntiva técnica fundamental en la arquitectura del software: modificar los pesos matemáticos internos del modelo mediante técnicas de Ajuste Fino (Fine-Tuning) para forzar la interiorización del conocimiento, o mantener los parámetros congelados y extender la memoria del modelo en tiempo real mediante la Generación Aumentada por Recuperación (RAG) consultando bases de datos documentales⁶.

La resolución de este dilema arquitectónico no puede fundamentarse en heurísticas derivadas del procesamiento del idioma inglés o de lenguas indoeuropeas de altos recursos. Las características inherentes a las lenguas morfológicamente ricas y aglutinantes, la presencia constante del translengüaje (alternancia de códigos) en las aulas bilingües y las estrictas limitaciones de memoria de acceso aleatorio (RAM) en el hardware local alteran radicalmente la viabilidad de ambos enfoques¹. El presente documento despliega una investigación comparativa exhaustiva sobre la viabilidad empírica, algorítmica y económica de RAG frente al

Ajuste Fino en SLMs, evaluando los costos computacionales de extremo a extremo y las métricas de precisión lingüística indispensables para mitigar la generación de información falsa (alucinaciones) en escenarios educativos críticos.

El Desafío Estructural: Morfología Aglutinante y el Cuello de Botella de la Tokenización

Para comprender el impacto computacional y lingüístico de cualquier arquitectura generativa sobre el Quechua Collao, es imperativo analizar primero la interacción entre la morfología intrínseca de la lengua y los algoritmos de tokenización estándar que anteceden a la red neuronal¹⁰. Los modelos de lenguaje modernos no procesan palabras como unidades discretas, sino *tokens*, que constituyen representaciones subléxicas generadas por algoritmos de compresión estadística como Byte-Pair Encoding (BPE) o SentencePiece¹¹. Estos algoritmos fueron diseñados y optimizados para lenguas analíticas, donde la frecuencia estadística de las subcadenas de caracteres se alinea de manera razonablemente coherente con las fronteras semánticas y sintácticas de las palabras¹³.

El Quechua Collao, por el contrario, es una lengua altamente aglutinante y de tendencia polisintética¹. En este sistema lingüístico, el significado se construye mediante la adición sucesiva de múltiples sufijos flexivos y derivativos a una raíz nominal o verbal. Este mecanismo permite la creación de palabras únicas de extrema complejidad semántica, las cuales equivaldrían a oraciones enteras en el idioma español¹. La aplicación de tokenizadores estadísticos convencionales sobre estas estructuras lingüísticas provoca un fenómeno ampliamente documentado en el estado del arte como el "cuello de botella de la tokenización" (Tokenization Bottleneck) o la Desalineación Token-Morfología (Tokenization-Morphology Misalignment, TMM)⁹.

Cuando un SLM recibe una secuencia en quechua, el tokenizador la fragmenta en múltiples piezas subléxicas arbitrarias que carecen de significado gramatical o morfológico individual, separando las raíces de sus sufijos portadores de sentido¹⁰. La literatura científica demuestra que esta fragmentación incrementa drásticamente la "fertilidad de tokens", definida como el número promedio de tokens necesarios para representar una sola palabra. Las evaluaciones empíricas en lenguas de estructura aglutinante similar revelan incrementos en la fertilidad de hasta un 3.4x en comparación con el inglés, lo que obliga al modelo a procesar secuencias artificialmente infladas para codificar el mismo contenido semántico⁹. A esto se suma la complejidad fonológica exclusiva de la variante Collao, que emplea consonantes oclusivas aspiradas y glotizadas (marcadas ortográficamente con apóstrofos o dígrafos), lo que confunde aún más a los tokenizadores entrenados en alfabetos latinos estándar, resultando en divisiones perjudiciales justo en el núcleo de las raíces verbales¹.

Implicaciones Directas en los Costos Computacionales de Inferencia

La fragmentación morfológica tiene ramificaciones matemáticas directas y severas en la eficiencia de la inferencia computacional. La complejidad algorítmica del mecanismo central de autoatención (self-attention) en la arquitectura Transformer crece de manera cuadrática con

respecto a la longitud de la secuencia de entrada, fenómeno expresado asintóticamente como $O(N^2 \cdot D)$, donde N representa el número de tokens y D la dimensionalidad del vector de incrustación (embedding)¹⁵.

Si la fertilidad de tokens en Quechua Collao triplica o cuadruplica la longitud de la secuencia en comparación con una consulta idéntica en español, la carga computacional demandada al hardware para procesar la misma instrucción pedagógica se multiplica de forma exponencial. Esto afecta de manera crítica el consumo de memoria asignada a la caché de claves-valores (KV Cache), la cual debe almacenar el estado de atención de cada token generado¹⁵.

Consecuentemente, el proceso ralentiza la velocidad de generación, provocando penalizaciones de latencia que pueden llegar a ser hasta 16.5 veces mayores en el procesamiento de secuencias de información equivalentes¹⁵. En dispositivos de borde (edge devices) o computadoras locales basadas exclusivamente en CPU, donde el ancho de banda de la memoria del sistema es el cuello de botella físico principal, procesar secuencias excesivamente largas debido a tokenizadores deficientes puede volver al sistema completamente inoperable en tiempo real¹⁵.

Ramificaciones Arquitectónicas para RAG y Ajuste Fino

El cuello de botella de la tokenización condiciona inevitablemente la viabilidad de la elección arquitectónica. En un escenario de Ajuste Fino, el modelo debe ajustar retroactivamente sus gradientes para aprender representaciones internas sobre subwords estadísticamente ruidosos y carentes de sentido aislado¹⁷. La evidencia sugiere que el entrenamiento sobre estos fragmentos, sin un alineamiento morfológico previo o una expansión profunda del vocabulario (vocabulary expansion), produce una memorización ineficiente⁹. El modelo gasta su limitada capacidad paramétrica intentando reconstruir la palabra original en lugar de interiorizar el conocimiento factual o pedagógico que se le intenta enseñar¹⁰.

En una arquitectura de Generación Aumentada por Recuperación (RAG), la altísima fertilidad de tokens consume rápidamente el límite máximo de la ventana de contexto (context window) del SLM¹⁹. Si un modelo compacto posee una capacidad de atención óptima de 2,048 o 4,096 tokens antes de comenzar a degradar su precisión o incurrir en costos inasumibles de memoria, inyectar tres o cuatro párrafos de documentos recuperados en quechua agotará este presupuesto de contexto de forma prematura¹⁹. Esto impone una restricción de ingeniería de software innegociable: el sistema de recuperación vectorial (retriever) debe ser excepcionalmente quirúrgico, extrayendo únicamente fragmentos semánticos (chunks) altamente relevantes y concisos para no asfixiar la capacidad de procesamiento del modelo generador²⁰.

Costos Computacionales en Hardware Restringido y Viabilidad del Producto Mínimo Viable

El pilar de la factibilidad del asistente educativo propuesto radica en su ejecución estrictamente fuera de línea (offline) en computadoras portátiles o dispositivos de bajas especificaciones (legacy hardware sin aceleración de GPU dedicada), tal como se requiere en las zonas rurales

del sur andino¹. En este ecosistema de infraestructura precaria, los costos de inferencia, la gestión de la memoria RAM y la latencia dictan el éxito pedagógico y la viabilidad técnica del despliegue⁹.

Para habilitar la inferencia eficiente de LLMs en CPU-only, el estándar tecnológico global se fundamenta en implementaciones en C/C++ altamente optimizadas, siendo llama.cpp y sus derivaciones el ecosistema dominante²². Esta biblioteca de software permite la compresión del modelo utilizando el formato GGUF (Generative GPT-Unified Format). Mediante la técnica de Cuantización Post-Entrenamiento (Post-Training Quantization, PTQ), se reduce la precisión de punto flotante de los tensores de peso originales de 16 bits (FP16 o BF16) a mapeos no lineales de 4 bits (e.g., el esquema Q4_K_M)¹⁶. Este proceso reduce el tamaño en disco y la huella en memoria del modelo en aproximadamente un 70%, permitiendo que un modelo de miles de millones de parámetros se aloje en la memoria del sistema (RAM) convencional¹⁹.

Sorprendentemente, esta compresión induce una degradación matemática marginal en la perplejidad del modelo, manteniendo intactas sus capacidades de razonamiento y comprensión lectora para tareas de nivel escolar¹⁹.

Basado en evaluaciones comparativas recientes (benchmarking) sobre hardware CPU estándar (como procesadores Intel Core i5/i7 o AMD Ryzen 5/7), la serie de modelos instruccionales pre-entrenados Qwen2.5 arroja perfiles de rendimiento marcadamente favorables para estos entornos, destacando por su robusto soporte multilingüe nativo que abarca más de 29 idiomas¹⁷.

Modelo Base e Instruccional	Formato de Cuantización	Tamaño en Disco (GGUF)	RAM Requerida en Inferencia	Velocidad Inferencia en CPU (Tokens/seg)	Viabilidad en Aulas EIB (Equipos Estándar)
Qwen2.5-3B	Q4_K_M	≈ 1.8 GB - 2.1 GB	≈ 2.5 GB - 3.2 GB	15 a 25 tok/s	Óptima (Laptops con 8 GB RAM) ²⁷
Qwen2.5-7B	Q4_K_M	≈ 4.4 GB - 4.9 GB	≈ 5.2 GB - 6.0 GB	8 a 15 tok/s	Viable (Laptops con 16 GB RAM) ²⁵
Qwen2.5-14B	Q4_K_M	≈ 8.5 GB - 9.0 GB	≈ 9.5 GB - 11 GB	3 a 6 tok/s	Inviabile (Latencia severa y OOM en

					8/16 GB) ¹⁹
--	--	--	--	--	------------------------

Nota: Los requerimientos de RAM contemplan el tamaño estático de los pesos más la asignación dinámica para la caché de contexto (KV Cache) durante interacciones largas.

El Impuesto Computacional de RAG en Tiempo de Ejecución

Mientras que el Ajuste Fino traslada prácticamente todo su inmenso costo computacional a la fase de laboratorio previa al despliegue, la arquitectura RAG impone un "impuesto" computacional dinámico durante la ejecución de cada consulta en el dispositivo del usuario final³.

El flujo RAG requiere recursos de procesamiento para operar el modelo generador de vectores (embedding model), ejecutar la búsqueda matemática de similitud en la base vectorial residente en la RAM y, lo que es más crítico para el rendimiento, forzar al SLM a leer todo el texto recuperado antes de poder emitir la primera sílaba de la respuesta³². Esta fase intensiva de lectura se denomina *prefill* (pre-llenado del contexto).

En el procesamiento basado en CPU, la velocidad del modelo se divide en dos fases con cuellos de botella distintos. El cálculo del *prefill* depende de la potencia de cómputo paralela (núcleos del procesador), mientras que la decodificación o generación de la respuesta token a token (decode) está severamente limitada por el ancho de banda de la memoria RAM (por ejemplo, la velocidad de transferencia de canales DDR4 o DDR5)²⁵. Si la base de datos RAG inyecta 2,000 tokens de contexto documental extraídos de manuales de la Academia Mayor de la Lengua Quechua, el procesador debe calcular la matriz de atención de esa secuencia masiva en un solo paso.

En herramientas altamente optimizadas directamente sobre el metal como llama-server, el procesamiento del *prefill* en CPU puede alcanzar entre 40 y 100 tokens por segundo, lo que significa que un contexto RAG moderado inducirá una latencia inicial conocida como Tiempo Hasta el Primer Token (Time-To-First-Token, TTFT) de 2 a 5 segundos²⁵. Sin embargo, la selección del ecosistema de software es vital: evaluaciones técnicas demuestran que *wrappers* populares como Ollama introducen sobrecargas de servidor que pueden ralentizar la decodificación entre un 38% y un 72% frente al uso directo de llama.cpp, y fallan estrepitosamente en el almacenamiento en caché del *prompt* (prompt caching) durante el RAG, provocando que el sistema vuelva a leer el documento entero en cada turno conversacional, elevando el TTFT a niveles inaceptables¹⁹.

Pese a esta latencia inherente, la investigación de Walusimbi et al. (2026) sobre arquitecturas de chatbots *offline-first* en entornos educativos de baja conectividad en el hemisferio sur concluye empíricamente que latencias de entre 1 y 5 segundos son toleradas positivamente por los usuarios en sistemas de aprendizaje asíncrono¹. Esta evidencia corrobora que, con la implementación optimizada de llama.cpp en su interfaz mínima nativa, el procesamiento RAG en hardware CPU es plenamente factible como producto mínimo viable (MVP)¹.

Evaluación Profunda del Ajuste Fino (Fine-Tuning) en SLMs

El Ajuste Fino (Fine-Tuning, FT) representa el proceso de entrenamiento supervisado continuo mediante el cual un modelo fundacional modifica sus parámetros internos utilizando un conjunto de datos específico del dominio, optimizando su función de pérdida matemática a través de algoritmos de propagación hacia atrás (backpropagation)³⁸. Su objetivo es interiorizar representaciones semánticas, inyectar conocimientos factuales, adoptar reglas gramaticales y replicar patrones estilísticos directamente en las matrices de pesos de las redes neuronales³.

Dadas las inmensas restricciones de hardware inherentes a los proyectos académicos y los despliegues locales, el Ajuste Fino completo (Full Fine-Tuning) —que actualiza el 100% de los miles de millones de parámetros del modelo— está fuera de los límites de la factibilidad económica³. En respuesta a esto, el estado del arte favorece el Ajuste Fino Eficiente en Parámetros (Parameter-Efficient Fine-Tuning, PEFT), estableciéndose la Adaptación de Bajo Rango (Low-Rank Adaptation, LoRA) como la metodología dominante⁴. LoRA mantiene congelados los inmensos pesos originales del modelo pre-entrenado e inyecta pequeñas matrices de descomposición de bajo rango en las capas de atención del Transformer. Esta técnica reduce el número de parámetros entrenables en más de un 99%, mitigando drásticamente los requerimientos de memoria de video (VRAM) durante la fase de entrenamiento y permitiendo ejecutar adaptaciones en hardware comercial de nivel superior². A pesar de las notables eficiencias teóricas aportadas por LoRA, el ajuste fino para inyectar conocimientos sigue siendo un proceso inherentemente intensivo en capital humano y computacional. Preparar un SLM para desempeñarse en el dominio de la EIB requiere la compilación rigurosa de miles de pares de preguntas y respuestas en formato de instrucción (Instruction Tuning) con una limpieza sintáctica y una veracidad semántica excepcionales¹. En lenguas aglutinantes de bajos recursos, la generación de esta data sintética de alta calidad conlleva un costo abrumador de validación por parte de lingüistas y hablantes nativos especializados⁴³.

El Riesgo Sistémico: Alucinaciones y Olvido Catastrófico

La ventaja indiscutible del ajuste fino radica en su insuperable capacidad para adaptar el estilo, el tono pedagógico y las convenciones morfológicas a las expectativas de la EIB⁶. Las evaluaciones demuestran que el FT sobresale en enseñar al modelo a generar respuestas que imitan fielmente el registro formal del currículo o el dialecto específico solicitado³. Sin embargo, para la tarea crítica de la inyección de conocimientos factuales —un requerimiento absoluto para un asistente educativo que debe proveer explicaciones científicas, geográficas o cívicas fundamentadas en el material escolar— el ajuste fino es arquitectónicamente deficiente y peligroso³. Múltiples estudios a gran escala dictaminan que forzar a un modelo compacto a memorizar hechos específicos y oscuros mediante la alteración de sus pesos puede desencadenar el "olvido catastrófico" (catastrophic forgetting), un fenómeno donde la sobreescritura de parámetros para acomodar la nueva información degrada de forma colateral el conocimiento general previo del modelo y erosiona sus capacidades lógicas fundamentales⁶.

Más crítico aún para el delicado ecosistema de la educación pública rural, el ajuste fino sobre

bases de datos limitadas exagera dramáticamente la propensión del modelo a generar alucinaciones sistémicas⁶. El SLM, actuando como una sofisticada máquina estadística, aprende a replicar a la perfección los patrones del corpus de entrenamiento, generando textos que poseen una "fluidez engañosa" (deceptive fluency)³⁸. Produce enunciados con una gramática quechua impecable y un tono sumamente autoritativo, pero que contienen afirmaciones factualmente incorrectas, conceptos inventados o, peor aún, que integran sesgos y estereotipos importados de su entrenamiento base en lenguas hegemónicas³⁸. El estudio paradigmático de Court y Elsner (2024) sobre el Quechua Sureño documenta exhaustivamente esta limitación³⁸. Sus experimentos demostraron que intentar inyectar reglas gramaticales extensas o exigir traducciones puramente paramétricas a los LLMs produce traducciones y aserciones infieles a nivel semántico, a menudo enmascaradas bajo una superficie lingüísticamente correcta que solo un especialista podría detectar como errónea³⁸. El ajuste fino interioriza el cómo hablar, pero no establece ningún límite verificable o barrera de seguridad algorítmica sobre la verdad absoluta de lo que se dice⁶.

Dinámicas del Translenguaje y Enrutamiento Lingüístico

En los ecosistemas educativos rurales del sur andino peruano, la interacción comunicativa no ocurre en compartimentos estancos monolingües. Docentes y estudiantes operan en un estado constante de bilingüismo dinámico, empleando la alternancia de códigos (code-switching) o translenguaje de manera orgánica y estratégica¹. Es habitual que un estudiante inicie una consulta en español, inserte terminología técnica, afectiva o cultural en Quechua Collao, y concluya la idea nuevamente en español⁵⁰.

Los LLMs y SLMs pre-entrenados sufren caídas catastróficas de rendimiento cuando se enfrentan a entradas (prompts) con lenguajes mixtos, particularmente cuando la alternancia involucra una lengua de dominio global y una lengua morfológicamente divergente de bajos recursos⁵⁰. En estas condiciones, el modelo generativo tiende a perder la coherencia contextual de su vector de atención, oscilando impredeciblemente entre gramáticas y produciendo una alta incidencia de invenciones subléxicas¹².

Para resolver este desafío algorítmico sin someter al modelo a un reentrenamiento masivo destinado a enseñarle a comprender el translenguaje nativo desde cero, la arquitectura requiere la orquestación de un sistema explícito de Identificación de Idioma (Language Identification, LID) a nivel de frase o token y un enrutamiento semántico previo a la fase de generación⁵¹. Las investigaciones recientes demuestran que el procesamiento de textos multilingües mixtos mejora notablemente cuando las arquitecturas de software disocian la fase de identificación de la fase de generación⁴³.

En la integración con una arquitectura RAG, la detección precisa de la alternancia de códigos permite al sistema aislar analíticamente los términos clave en cada idioma y ejecutar consultas de recuperación duales. Por ejemplo, el sistema puede rastrear un concepto normativo en la base de datos del Currículo Nacional indexada en español, simultáneamente buscar el concepto cultural asociado en el corpus paralelo de la Academia Mayor de la Lengua Quechua,

e inyectar ambos contextos de manera estructurada en el *prompt* final dirigido al SLM¹. Esta decisión arquitectónica traslada la inmensa responsabilidad cognitiva de interpretar el translenguaje desde los opacos parámetros internos del SLM hacia una capa de software de recuperación algorítmica robusta, determinista y auditable³².

Generación Aumentada por Recuperación (RAG) en SLMs: Mitigación de Vulnerabilidades

La Generación Aumentada por Recuperación (RAG) representa un salto cualitativo hacia la veracidad y la transparencia: la arquitectura desacopla explícitamente el conocimiento factual de los parámetros probabilísticos del modelo³. El sistema congela los pesos del SLM subyacente y delega el almacenamiento de la información del mundo real a una base de datos vectorial externa⁶. Durante la inferencia, una consulta del usuario se vectoriza mediante un modelo codificador (embedding model), se calcula la similitud geométrica contra una colección pre-indexada de documentos EIB, y los textos más relevantes se extraen para ser concatenados en la ventana de contexto del SLM³.

Control Determinista de la Veracidad Factual

La evidencia científica contemporánea posiciona unánimemente a RAG como la arquitectura superior y ética para aplicaciones de dominio específico que exigen fidelidad absoluta a fuentes oficiales⁶. Revisiones sistemáticas en dominios críticos revelan que RAG reduce las tasas de alucinación clínica o técnica entre un 12% y un 53%, aplastando de manera consistente el rendimiento de los modelos sometidos exclusivamente a ajuste fino paramétrico⁶.

La supremacía del enfoque RAG se magnifica radicalmente al procesar entidades de baja frecuencia (low-frequency entities)³⁷. En el caso del asistente propuesto, nociones endémicas del pensamiento andino, neologismos pedagógicos en Quechua Collao o artículos específicos de los manuales del MINEDU constituyen entidades de escasa representación en el pre-entrenamiento global de cualquier modelo fundacional¹. RAG asegura algorítmicamente que cuando el estudiante formule una pregunta, el SLM anclará (grounding) su respuesta directamente en los textos convalidados por la Academia Mayor de la Lengua Quechua y los corpus paralelos establecidos por investigaciones como las de Orcotoma Mormontoy et al. (2025)¹. Esta restricción previene extrapolaciones y dota al sistema educativo de trazabilidad absoluta, garantizando que el origen de cada explicación pueda ser auditado hasta el párrafo fuente³.

Vulnerabilidades Estructurales de RAG en SLMs

Si bien los beneficios hacia la veracidad son axiomáticos, la aplicación de flujos RAG estándar sobre modelos compactos introduce vulnerabilidades arquitectónicas que obligan a reconsiderar la ingenuidad de las implementaciones por defecto (RAG Naïve). Los LLMs masivos poseen capacidades atencionales extraordinarias para discernir señales relevantes del ruido dentro de un *prompt* extenso. Los SLMs, en dramático contraste, son inherentemente susceptibles a la distracción semántica y al colapso por saturación de contexto²⁰.

Un estudio crítico conducido por Hevia et al. (2025) en el desarrollo de tutores de inteligencia artificial fuera de línea demostró empíricamente que la inclusión de un flujo RAG estándar puede degradar significativamente la precisión general de los SLMs más compactos (como la familia SmolLM)²⁰. Cuando los fragmentos de texto recuperados (chunks) introducen información ruidosa, redundante o periférica, el modelo pequeño fracasa al localizar la respuesta correcta, sufriendo el fenómeno de "atención diluida"²¹. Esta limitación intrínseca de los SLMs exige que el diseño de ingeniería del módulo de recuperación de la tesis sea excepcionalmente refinado. La vectorización estándar de párrafos fijos es insuficiente; resulta indispensable instrumentar un Fragmentado Semántico (Semantic Chunking) que delimite los textos curriculares respetando estrictamente la cohesión lógica y los límites de las ideas²¹. Las métricas de recuperación como la Precisión del Contexto (Context Precision) del marco de evaluación RAGAS (Retrieval-Augmented Generation Assessment) adquieren un peso preponderante sobre las métricas de generación⁵⁷. Solo un motor de búsqueda meticulosamente depurado, que retorne exclusivamente uno o dos fragmentos de alta densidad semántica (k=1 o k=2), permitirá al modelo Qwen2.5-3B capitalizar el aumento de conocimiento sin verse sobrepasado por la carga cognitiva²¹.

Dimensión de Análisis	Ajuste Fino Eficiente (LoRA)	Generación Aumentada por Recuperación (RAG)	Veredicto para Asistencia EIB
Mitigación de Alucinaciones	Baja. Tendencia a la fluidez engañosa en datos desconocidos.	Altísima. Reducción comprobada del 12-53% al forzar el anclaje textual.	RAG es imperativo para la seguridad pedagógica ⁶ .
Costo de Actualización Curricular	Alto. Requiere reiniciar ciclos intensivos de entrenamiento GPU para cada cambio.	Nulo. Basta con eliminar o añadir archivos a la base vectorial local.	RAG provee viabilidad operativa asíncrona ³ .
Adaptación Estilística y Morfológica	Alta. El modelo aprende a mimetizar profundamente la gramática y el tono.	Baja. El modelo depende de la capacidad subyacente que ya poseía.	Ajuste Fino es óptimo para internalizar sintaxis quechua ³ .
Costos de	Nulos. Genera	Altos. Penalización	Ajuste Fino es

Inferencia Local (CPU)	respuestas a la máxima velocidad nominal del hardware.	TTFT de 2-5 segundos debido al prefill del documento extra.	marginalmente más eficiente, aunque RAG es tolerable (1-5s) ¹ .
------------------------	--	---	--

Sinergias Emergentes: RAFT y el Precedente del Modelo QueEn

Es crucial comprender que en las fronteras de la investigación en procesamiento de lenguaje natural de 2024-2026, la Generación Aumentada por Recuperación y el Ajuste Fino ya no se consideran metodologías mutuamente excluyentes, sino ejes ortogonales de optimización que confluyen en arquitecturas híbridas³.

La metodología más disruptiva en este panorama es el **Ajuste Fino Aumentado por Recuperación (Retrieval-Augmented Fine-Tuning, RAFT)**⁶⁷. A diferencia del ajuste fino convencional que intenta enseñar hechos enciclopédicos al modelo, RAFT entrena al modelo específicamente en la *habilidad procedimental* de ejecutar el flujo RAG de manera resiliente⁶⁹. Durante la creación del corpus de entrenamiento RAFT, se suministran consultas emparejadas con documentos donde una fracción contiene el "oráculo" (la respuesta correcta) rodeado de ruido, y otra fracción contiene exclusivamente "documentos distractores" irrelevantes⁶⁷. El modelo se entrena algorítmicamente para ignorar el ruido distractor, extraer citas textuales exactas del material relevante y, fundamentalmente, negarse a responder basándose en su conocimiento paramétrico cuando el documento provisto no contenga la respuesta⁶⁸. Esta técnica ha demostrado vacunar a los modelos instruccionales frente a las imperfecciones de los sistemas de recuperación, maximizando la robustez general⁶⁷.

En el contexto específico y directo del idioma quechua, el desarrollo del sistema **QueEn (Quechua-English Translation)** por Chen et al. solidifica la viabilidad de la hibridación⁴. Para superar la escasez de recursos digitales y la profunda complejidad polisintética del quechua, los investigadores arquitectaron una solución donde RAG operaba sobre diccionarios externos y recursos lingüísticos dinámicos, mientras simultáneamente aplicaban la adaptación LoRA para moldear la comprensión interna del modelo⁴. Los resultados de este enfoque dual fueron extraordinarios: la sinergia catapultó la precisión de la traducción, elevando el puntaje de la métrica BLEU desde un inoperable 1.5 en el modelo GPT base, hasta un impresionante 17.6⁴. Este precedente establece un paradigma validado científicamente: la recuperación vectorial acoplada a la adaptación paramétrica ligera conforma un multiplicador exponencial para el dominio de lenguas en peligro⁴.

Conclusiones para la Arquitectura del Producto Mínimo Viable

La integración exhaustiva de la literatura especializada y la evidencia técnica de rendimiento empírico validan sólidamente la dirección arquitectónica trazada para el asistente educativo

offline de la UNSAAC. Abordar la brecha lingüística y digital mediante SLMs cuantizados no es solo un planteamiento innovador, sino la única estrategia tecnológica sustentable para este entorno geográfico.

La síntesis de los hallazgos impone las siguientes definiciones para la investigación:

En primer lugar, la elección del ecosistema de ejecución debe descartar implementaciones de alto nivel que introduzcan latencias innecesarias (como el prefill ineficiente de Ollama) en favor de ejecuciones nativas de alto rendimiento (como llama-server) empleando la familia Qwen2.5 de 3B a 7B parámetros²⁵. La compresión GGUF en el formato Q4_K_M provee el punto de equilibrio inmejorable entre ocupación de RAM y conservación de la inteligencia generativa para asegurar la ejecución en hardware de 8 GB a 16 GB¹⁶.

En segundo lugar, el dilema central se decanta de forma incontestable a favor de la Generación Aumentada por Recuperación (RAG) como pilar del Producto Mínimo Viable (MVP). La morfología aglutinante del quechua impone una severa penalización en la fragmentación de tokens, lo que vuelve arriesgado y computacionalmente destructivo forzar a un SLM a memorizar currículos locales mediante ajuste fino⁹. RAG soluciona la necesidad crítica de seguridad escolar al restringir mecánicamente al modelo a derivar sus explicaciones de los textos controlados de la AMLQ y el MINEDU, abatiendo las alucinaciones sistemáticas derivadas del entrenamiento deficiente del modelo base en identidades de baja frecuencia cultural⁶.

Finalmente, el diseño de la investigación debe reconocer las limitaciones inherentes a los modelos pequeños y la complejidad del translenguaje. La fase de ingeniería lógica del RAG exigirá un esfuerzo preponderante en el enrutamiento previo del idioma (Language Routing) y en la aplicación de un fragmentado semántico minucioso para proteger al modelo de la distracción por ruido contextual²¹. Una vez que este cimientista determinista haya probado su eficacia empírica en el terreno bajo métricas lingüísticas robustas (como chrF++ y Context Precision), el proyecto estará posicionado inmejorablemente para evolucionar hacia arquitecturas híbridas de vanguardia (RAFT/LoRA), sellando una contribución fundacional y tecnológicamente pragmática a la educación andina³.

Obras citadas

1. PlanTesis_SLM_QuechuaCollao.pdf
2. Small Language Models (SLMs) | Uniphore,
<https://www.uniphore.com/insights/small-language-models-slms/>
3. Full Guide to Choosing the Right AI Stack. Part 1: RAG vs. Fine-Tuning vs. Hybrid,
<https://intersog.co.il/blog/full-guide-to-choosing-the-right-ai-stack-part-1-rag-vs-fine-tuning-vs-hybrid/>
4. QueEn: A Large Language Model for Quechua-English Translation - arXiv,
<https://arxiv.org/html/2412.05184v1>
5. Meet QueEn, a Large Language Model for Quechua-English Translation - Slator,
<https://slator.com/meet-queen-large-language-model-for-quechua-english-translation/>
6. Retrieval-Augmented Generation (RAG) vs. Fine-Tuning: A Systematic Review of

- Hallucination Mitigation in Medical LLMs | Authorea,
<https://www.authorea.com/doi/10.22541/au.176903295.54191070>
7. RAG vs Fine-Tuning: How to Adapt LLMs to Your Data - Digital Sense,
<https://www.digitalsense.ai/blog/fine-tuning-vs-rag>
 8. RAG vs Fine-Tuning: A Developer's Guide to Enhancing AI Performance - Reddit,
https://www.reddit.com/r/Rag/comments/1izifif/rag_vs_finetuning_a_developers_guide_to_enhancing/
 9. Beyond the Tokenization Bottleneck: A Conceptual Framework for Efficiency-Plasticity Trade-Offs in ASEAN Large Language Model Adaptation | Cureus Journals,
<https://www.cureusjournals.com/articles/13958-beyond-the-tokenization-bottleneck-a-conceptual-framework-for-efficiency-plasticity-trade-offs-in-asean-large-language-model-adaptation>
 10. Correcting the Tokenization Bias in XAI via Morphologically-Aligned Projection - ACL Anthology, <https://aclanthology.org/2026.sigturk-1.19.pdf>
 11. Neural Machine Translation of Rare Words with Subword Units - ResearchGate,
https://www.researchgate.net/publication/306093632_Neural_Machine_Translation_of_Rare_Words_with_Subword_Units
 12. Tokenization is Killing our Multilingual LLM Dream - Hugging Face,
<https://huggingface.co/blog/omarkamali/tokenization>
 13. Syllabic Agglutinative Tokenizations for Indonesian LLM: A Study from “Gasing Literacy Learning System” - arXiv, <https://arxiv.org/html/2601.11643v1>
 14. Tokenization Strategies for Low-Resource Agglutinative Languages in Word2Vec: Case Study on Turkish and Finnish - ResearchGate,
https://www.researchgate.net/publication/395649028_Tokenization_Strategies_for_Low-Resource_Agglutinative_Languages_in_Word2Vec_Case_Study_on_Turkish_and_Finnish
 15. Tokenization Impacts Multilingual Language Modeling: Assessing Vocabulary Allocation and Overlap Across Languages | Request PDF - ResearchGate,
https://www.researchgate.net/publication/372915763_Tokenization_Impacts_Multilingual_Language_Modeling_Assessing_Vocabulary_Allocation_and_Overlap_Across_Languages
 16. Llamas on the Web: Memory-Efficient, Performance-Portable, and Multi-Precision LLM Inference with WebGPU - arXiv, <https://arxiv.org/html/2605.20706v1>
 17. Qwen2.5-3B-Instruct-GGUF - 模型详情页,
<https://modelscope.cn/models/qwen/Qwen2.5-3B-Instruct-GGUF>
 18. How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models | Request PDF - ResearchGate,
https://www.researchgate.net/publication/353491635_How_Good_is_Your_Tokenizer_On_the_Monolingual_Performance_of_Multilingual_Language_Models
 19. Local LLM Speed: RTX 3060, Qwen2 & Llama Benchmark Results - Ajit Singh,
<https://singhajit.com/llm-inference-speed-comparison/>
 20. Towards an Efficient, Customizable, and Accessible AI Tutor - arXiv,
<https://arxiv.org/pdf/2510.06255>
 21. Towards an Efficient, Customizable, and Accessible AI Tutor - GitHub,

- <https://raw.githubusercontent.com/mlresearch/v273/main/assets/hevia25a/hevia25a.pdf>
22. Deploying LLMs on CPU-only Environments with llama.cpp Library Set: MedLocalGPT Project Case - CEUR-WS.org, <https://ceur-ws.org/Vol-4164/paper11.pdf>
 23. An Open-Source Multilingual RAG Pipeline for Policy Analysis - WebThesis - PoliTO, <https://webthesis.biblio.polito.it/39995/1/tesi.pdf>
 24. llama.cpp - Qwen - Read the Docs, https://qwen.readthedocs.io/en/latest/run_locally/llama.cpp.html
 25. How fast is LlamaStash? Overhead, throughput, and a fair comparison with Ollama and LM Studio | Technorage - Deepu K Sasidharan, <https://deepu.tech/benchmarking-llamastash/>
 26. Qwen 2.5 CPU vs GPU comparison : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1fq883g/qwen_25_cpu_vs_gpu_comparison/
 27. keon/awesome-nlp: :book: A curated list of resources dedicated to Natural Language Processing (NLP) - GitHub, <https://github.com/keon/awesome-nlp>
 28. novaforgeai/qwen2.5-3b - Ollama, <https://ollama.com/novaforgeai/qwen2.5-3b>
 29. AI on 16GB RAM: Every Model You Can Actually Run (2026 Tested), <https://localaimaster.com/blog/ai-models-16gb-ram>
 30. Distributed LLM Inference: llama.cpp RPC across Metal + CUDA over 10GbE - GitHub, <https://github.com/kjaiswal/llama-cpp-distributed-benchmarks>
 31. Run LLMs Locally on ARM: Ollama & llama.cpp on RK3588 [2026] - Turing Pi, <https://turingpi.com/run-llm-locally-arm-rk3588-ollama-llama-cpp/>
 32. (PDF) Advancing Retrieval-Augmented Generation (RAG) Innovations, Challenges, and the Future of AI Reasoning - ResearchGate, https://www.researchgate.net/publication/388722115_Advancing_Retrieval-Augmented_Generation_RAG_Innovations_Challenges_and_the_Future_of_AI_Reasoning
 33. Towards an Efficient, Customizable, and Accessible AI Tutor - ResearchGate, https://www.researchgate.net/publication/396331435_Towards_an_Efficient_Customizable_and_Accessible_AI_Tutor
 34. benchmarks? · Issue #34 · ggml-org/llama.cpp - GitHub, <https://github.com/ggml-org/llama.cpp/issues/34>
 35. MoE on CPU: 13B-Class Answers at 3B Speed - Inventive HQ, <https://inventivehq.com/blog/moe-on-cpu-benchmark>
 36. Deploying LLM on IPHONE using LLAMA.CPP and metal - Fabricio Batista Narcizo, https://www.fabricionarcizo.com/cvpr2026-edge-ai-in-action/pdfs/cvpr2026/05-LLM_Models_on_iPhone_using_Llama-CPP.pdf
 37. Fine Tuning vs. Retrieval Augmented Generation for Less Popular Knowledge - arXiv, <https://arxiv.org/html/2403.01432v5>
 38. Especialización de LLM: hacia una optimización por tarea y ámbito - UPM[Blogs], <https://blogs.upm.es/catedra-idanae/wp-content/uploads/sites/698/2026/01/Idanae-4T25.pdf>
 39. Boosting GPT models for genomics analysis: generating trusted genetic variant

- annotations and interpretations through RAG and Fine-tuning | Bioinformatics Advances | Oxford Academic,
<https://academic.oup.com/bioinformaticsadvances/article/5/1/vbaf019/8002096>
40. richardyoung/qwen2.5-7b-instruct-abliterated:Q4_K_M - Ollama,
https://ollama.com/richardyoung/qwen2.5-7b-instruct-abliterated:Q4_K_M
 41. AI-Driven Generation of Old English: A Framework for Low-Resource Languages - MDPI, <https://www.mdpi.com/2504-2289/10/5/145>
 42. [Literature Review] QueEn: A Large Language Model for Quechua-English Translation,
<https://www.themoonlight.io/en/review/queen-a-large-language-model-for-quechua-english-translation>
 43. Multilingual NLP: Code-Switching, Variants, & Dialectal Expansion - Appen,
<https://www.appen.com/blog/emnlp-2025-multilingual-nlp>
 44. Fine-tuning LLMs for Enterprise RAG — A design perspective | by Raunak Jain | Medium,
<https://medium.com/@raunak-jain/fine-tuning-llms-for-enterprise-rag-8c1eb3ac6b32>
 45. (PDF) Medical LLMs: Fine-Tuning vs. Retrieval-Augmented Generation - ResearchGate,
https://www.researchgate.net/publication/393000121_Medical_LLMs_Fine-Tuning_vs_Retrieval-Augmented_Generation
 46. the ability of ChatGPT to create texts in Andalusian dialect of Spanish language - UEF eRepo,
<https://erepo.uef.fi/bitstreams/8ba8dbe6-029a-47e2-983d-46fd2cf013f1/download>
 47. BLEU scores for all LLMs and prompt types. | Download Scientific Diagram - ResearchGate,
https://www.researchgate.net/figure/BLEU-scores-for-all-LLMs-and-prompt-types_tbl2_386184219
 48. An Empirical Study of Many-Shot In-Context Learning for ... - Bytez,
<https://bytez.com/docs/arxiv/2604.02596/paper>
 49. (PDF) Bridging Languages in Healthcare: A Comprehensive Review of Multilingual and Code-Switched Chatbot Interactions - ResearchGate,
https://www.researchgate.net/publication/400737848_Bridging_Languages_in_Healthcare_A_Comprehensive_Review_of_Multilingual_and_Code-Switched_Chatbot_Interactions
 50. Beyond Monolingual Assumptions: A Survey on Code-Switched NLP in the Era of Large Language Models across Modalities - arXiv,
<https://arxiv.org/html/2510.07037v4>
 51. How does NLP handle code-switching in multilingual texts? - Milvus,
<https://milvus.io/ai-quick-reference/how-does-nlp-handle-codeswitching-in-multilingual-texts>
 52. Code-Switching Reveals Language Anchoring in Multilingual LLMs - arXiv,
<https://arxiv.org/html/2606.19668>
 53. Language Identification of Code Switching Sentences and Multilingual Sentences

- of Under-Resourced Languages by Using Multi Struc - ISCA Archive,
https://www.isca-archive.org/interspeech_2014/yeong14_interspeech.pdf
54. CollectivIA: Two-Pipeline Multilingual Legal RAG for Moroccan Territorial Governance with LLM-Assisted and Regex-Based Chunking - MDPI,
<https://www.mdpi.com/2504-2289/10/7/212>
 55. Towards the Creation of a Collao Quechua–Spanish Parallel Corpus Using Optical Character Recognition - ACL Anthology,
<https://aclanthology.org/2025.globalnlp-1.1/>
 56. Building Custom AI Applications for Multilingual Support Desks | MindStudio,
<https://www.mindstudio.ai/blog/custom-ai-applications-multilingual-support-desks>
 57. advancing retrieval-augmented - arXiv, <https://arxiv.org/pdf/2501.04858>
 58. Leveraging Retrieval-Augmented Generation for Swahili Language Conversation Systems, <https://www.mdpi.com/2076-3417/15/2/524>
 59. Adapting Large Language Models for Arabic: Comparative Evaluation, Fine-Tuning, and Ethical Deployment - ResearchGate,
https://www.researchgate.net/publication/396722609_Adapting_Large_Language_Models_for_Arabic_Comparative_Evaluation_Fine-Tuning_and_Ethical_Deployment
 60. Fine Tuning vs. Retrieval Augmented Generation for Less Popular Knowledge - arXiv, <https://arxiv.org/html/2403.01432v1>
 61. Proceedings of the Workshop on Beyond English: Natural Language Processing for all Languages in an Era of Large Language Models - ACL Anthology,
<https://aclanthology.org/2025.globalnlp-1.0.pdf>
 62. Augmented Generation and Semantic Chunking - IJRASET,
<https://www.ijraset.com/best-journal/optimizing-crosslingual-information-retrieval-in-campus-administration-using-retrieval-augmented-generation-and-semantic-chunking>
 63. Fine-tuning Large Language Models with Domain-Specific Data for Retrieval-Augmented Question-Answering Systems - Aaltodoc,
<https://aaltodoc.aalto.fi/bitstreams/3d8dbe30-d954-409b-8b57-3781013b3ecd/download>
 64. Efficient Knowledge Injection in LLMs via Self-Distillation - arXiv,
<https://arxiv.org/html/2412.14964v2>
 65. [2510.06255] Towards an Efficient, Customizable, and Accessible AI Tutor - arXiv,
<https://arxiv.org/abs/2510.06255>
 66. Fine-Tuning Retrieval-Augmented Generation with an Auto-Regressive Language Model for Sentiment Analysis in Financial Reviews - MDPI,
<https://www.mdpi.com/2076-3417/14/23/10782>
 67. RAFT: Adapting Language Model to Domain Specific RAG - Gorilla,
https://gorilla.cs.berkeley.edu/blogs/9_raft.html
 68. Paper page - RAFT: Adapting Language Model to Domain Specific RAG - Hugging Face,
<https://huggingface.co/papers/2403.10131>
 69. RAFT: Adapting Language Model to Domain Specific RAG - arXiv,
<https://arxiv.org/html/2403.10131v2>

70. RAFT: Adapting Language Model to Domain Specific RAG - Santosh Sawant, <https://ssawant.github.io/posts/RAFT/RAFT.html>
71. [2403.10131] RAFT: Adapting Language Model to Domain Specific RAG - arXiv, <https://arxiv.org/abs/2403.10131>
72. RAFT: Adapting Language Model to Domain Specific RAG - OpenReview, <https://openreview.net/forum?id=rzQGHXNReU-eld=i4QAZSOS5r>
73. QueEn: A Large Language Model for Quechua-English Translation - ChatPaper, <https://chatpaper.com/chatpaper/paper/88170>
74. QueEn: A Large Language Model for Quechua-English Translation - ResearchGate, https://www.researchgate.net/publication/386555117_QueEn_A_Large_Language_Model_for_Quechua-English_Translation
75. [2412.05184] QueEn: A Large Language Model for Quechua-English Translation - arXiv, <https://arxiv.org/abs/2412.05184>